

Sistemas Distribuidos
GRADO EN INGENIERÍA INFORMÁTICA



MEMORIA

Ejercicio evaluable 3: Sistemas Distribuidos

Curso 2025/2026

DATOS DE LOS AUTORES

Nombre:	David Sierra
NIA:	100471913
Correo electrónico:	100471913@alumnos.uc3m.es

Nombre:	Juan Luis del Valle Caetano
NIA:	100454201
Correo electrónico:	100454201@alumnos.uc3m.es

1. Introduccion

Este tercer ejercicio evaluable extiende el trabajo realizado en los ejercicios anteriores convirtiendo el servicio de almacenamiento de tuplas en un sistema distribuido basado en ONC RPC (Open Network Computing Remote Procedure Call). Mientras que el ejercicio 2 implementaba la distribucion mediante sockets TCP con un protocolo propio de texto, en este ejercicio se utiliza el estandar de RPC que automatiza la serializacion de datos, el transporte y el binding mediante la herramienta rpcgen y la biblioteca TI-RPC.

El objetivo es mantener la misma interfaz publica definida en claves.h (funciones destroy, set_value, get_value, modify_value, delete_key y exist) sin modificar el codigo cliente ni la implementacion local del servicio, cambiando unicamente la capa de transporte.

2. Arquitectura del Sistema

El sistema se compila en dos modos de operacion independientes:

- **Version local (Parte A):** la aplicacion app-cliente enlaza directamente con libclaves.so. El servicio reside en el mismo proceso y las llamadas son locales.
- **Version distribuida (Parte B):** la aplicacion app-cliente-rpc enlaza con libproxyclaves.so, que hace llamadas RPC sobre TCP al servidor clavesRPC_server, el cual internamente usa la misma libclaves.so.

La aplicacion cliente (app-cliente.c) es identica en ambas versiones; unicamente cambia la biblioteca con la que se enlaza. Esto demuestra que la interfaz claves.h actua como contrato estable entre cliente y servidor, sin que la logica de pruebas tenga conocimiento de si las llamadas son locales o remotas.

Fichero	Rol
claves.h	Interfaz publica del servicio (sin modificar desde Ej. 1)
claves.c	Implementacion local: lista enlazada con mutex POSIX
clavesRPC.x	Definicion del interfaz RPC en lenguaje XDR
clavesRPC_server.c	Handlers del servidor RPC: adaptan tipos XDR a tipos locales
proxy-rpc.c	Implementacion de claves.h via llamadas RPC (libproxyclaves.so)
app-cliente.c	Plan de pruebas reutilizado del Ejercicio 1
Makefile	Compilacion: version local y distribuida

3. Definicion del Interfaz RPC (clavesRPC.x)

El fichero .x es el punto de partida del sistema distribuido. Define los tipos de datos transferibles y los procedimientos remotos en lenguaje XDR. La herramienta rpcgen procesa este fichero y genera automaticamente el codigo de serializacion XDR, el stub del servidor (clavesRPC_svc.c) y el stub del cliente (clavesRPC_clnt.c), liberando al programador de implementar la capa de comunicacion a mano.

3.1 Tipos de datos

Tipo	Descripcion
PaqueteRPC	Equivalente a struct Paquete de claves.h. Tres enteros (x, y, z).
VectorFloat<32>	Vector de floats de longitud variable (max. 32 elementos). XDR serializa la longitud y los datos.
SetModifyArgs	Agrupar los cinco parametros de set_value y modify_value, ya que ONC RPC solo admite un argumento por procedimiento.
GetValueResult	Respuesta de get_value: codigo de resultado mas los tres valores (value1, N_value2/V_value2, value3).

3.2 Procedimientos remotos

Se definen seis procedimientos en el programa CLAVES_PROG (identificador 0x20000001, rango reservado para aplicaciones de usuario), version 1:

Procedimiento	Argumento	Resultado
DESTROY (1)	void	int
SET_VALUE (2)	SetModifyArgs	int
GET_VALUE (3)	string (clave)	GetValueResult
MODIFY_VALUE (4)	SetModifyArgs	int
DELETE_KEY (5)	string (clave)	int
EXIST (6)	string (clave)	int

4. Implementacion

4.1 Servidor RPC (clavesRPC_server.c)

El servidor implementa las seis funciones _1_svc generadas por rpcgen. Cada funcion actua exclusivamente como adaptador de tipos: convierte los tipos XDR a los tipos nativos de claves.h y delega en la implementacion local de claves.c. No contiene logica de negocio propia.

La operacion mas delicada es get_value_1_svc, ya que el resultado incluye buffers de memoria dinamica que deben permanecer validos hasta que XDR los serialice. Se reserva memoria con malloc y el sistema XDR la libera automaticamente mediante claves_prog_1_freeresult, que llama a xdr_free. En caso de error se inicializan igualmente los punteros con cadenas vacias para evitar punteros NULL que XDR no puede serializar.

4.2 Proxy cliente (proxy-rpc.c)

El proxy implementa la interfaz claves.h realizando llamadas RPC al servidor. La direccion del servidor se lee de la variable de entorno IP_TUPLAS en tiempo de ejecucion. El flujo de cada operacion sigue un patron uniforme:

1. Validar parametros localmente (evita crear conexiones para peticiones invalidas).
2. Leer IP_TUPLAS y crear un cliente RPC TCP con clnt_create.
3. Construir la estructura de argumentos XDR y llamar al stub RPC generado.
4. Comprobar el estado de la llamada (enum clnt_stat). Si falla, devolver -2.
5. Destruir el cliente con clnt_destroy y devolver el resultado al llamador.

5. Decisiones de Diseno

5.1 Conexion RPC por operacion

El proxy crea y destruye el cliente RPC en cada llamada en lugar de mantener un handle CLIENT* persistente. Esta decision simplifica la gestion del ciclo de vida y evita problemas de reconexion si el servidor se reinicia. El coste es la latencia adicional del handshake TCP en cada operacion, asumible en un contexto academico donde prima la simplicidad.

5.2 Protocolo TCP sobre UDP

Se usa TCP como protocolo de transporte. TCP garantiza entrega fiable y orden de mensajes, lo cual es necesario porque operaciones como set_value, modify_value o delete_key no son idempotentes: repetirlas ante un timeout de UDP podria corromper el estado del servicio (por ejemplo, duplicar una insercion o borrar una clave recién creada).

5.3Codigo de error -2 para fallos de red

El proxy devuelve -2 cuando hay un fallo de comunicacion (servidor no disponible o llamada RPC fallida) y -1 para errores de aplicacion (clave duplicada, parametros fuera de rango). Esto permite al cliente distinguir entre un error logico y un fallo de infraestructura, aunque la interfaz original claves.h solo contemplaba -1 como codigo de error generico.

5.4 Reutilizacion integral de claves.c

La implementacion local del ejercicio 1, basada en lista enlazada con mutex POSIX, se reutiliza sin modificar una sola linea. El servidor RPC actua unicamente como capa de traduccion entre el mundo XDR y la API local, lo que valida el diseno de claves.h como abstraccion estable e independiente del mecanismo de distribucion.

5.5 Estructura SetModifyArgs compartida

ONC RPC solo permite pasar un unico argumento por procedimiento. Como set_value y modify_value necesitan cinco parametros, se agrupan en SetModifyArgs. Al ser identicos los parametros de ambas operaciones, se reutiliza la misma estructura, evitando duplicidad en el fichero .x.

6. Conclusiones

ONC RPC simplifica significativamente el desarrollo de sistemas distribuidos respecto al uso directo de sockets. La herramienta rpcgen automatiza la serializacion XDR y el codigo de infraestructura (registro en el portmapper, dispatch de llamadas), permitiendo centrarse en la logica de negocio. El resultado es un codigo del servidor mas limpio y mantenible que el equivalente con sockets del ejercicio 2.

La mayor dificultad encontrada fue la gestion correcta de la memoria dinamica en la operacion get_value del servidor: XDR requiere que los punteros en las estructuras de respuesta sean validos en el momento de la serializacion, y el propio framework se encarga de liberarlos mediante xdr_free. Entender este ciclo de vida fue el aspecto mas critico de la implementacion.